

Acara 9 (Pemrograman Dart II (Function))

Materi Pembelajaran : Pemrograman Dart II
Acara Praktikum/Praktik : Pemrograman Dart II (Function)
Tempat : *Laboratorium Multimedia Cerdas*
Alokasi Waktu : *170 Menit*

a. Capaian Pembelajaran Mata Kuliah (CPMK) :

- Mahasiswa mampu memahami pemrograman dart.
- Mahasiswa mampu mengenal struktur function dalam pemrograman dart.

b. Penilaian Bertahap BNSP (*Skill Passport*) :

Nama Skema Sertifikasi :

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
	J.612000.021	Mengembangkan Mobile Sensor pada Mobile Computing Environment	Menjelaskan pemrograman sensor Mobile

c. Indikator Penilaian :

- Keberhasilan mahasiswa dalam menggunakan function pada studi kasus.
- Keberhasilan mahasiswa dalam menggunakan function pada studi kasus.

d. Dasar Teori :

Function adalah sebuah blok kode yang disusun sedemikian rupa untuk menjalankan sebuah tindakan. Blok kode ini dibuat untuk dapat bisa digunakan kembali. Cara atau bentuk penulisan function adalah sebagai berikut:

```
function nama_function(parameter 1, parameter 2, ...) {  
  [Isi dari function berupa tindakan]  
  return [expression];  
}
```

Kode di atas tidak dapat kita copy-paste kan langsung, melainkan hanya sebuah bentuk penulisan **function**. Sebuah **function**, umumnya melakukan tindakan dan sebelum **function** berakhir, **function** bisa mengembalikan nilai dengan cara menambahkan sintaks return.

Kita juga dapat mengirimkan nilai ke dalam sebuah **function** dengan mencantumkan ke dalam tanda kurung dalam penulisan **function**. Untuk mengirimkan nilai lebih dari satu, gunakan tanda **,** sebagai pemisah.

e. Alat dan Bahan :

Adapun alat dan bahan meliputi:

- Kertas HVS
- Spidol Besar
- Bolpoin

f. Prosedur Kerja ;

Contoh Function 1: Function sederhana tanpa return

```
void main(){
    tampilkan();
}

tampilkan(){
    print("Hello Peserta Bootcamp");
}
```

Contoh Function 2: Function sederhana dengan return

```
void main(){
    print(munculkanangka());
}

munculkanangka(){
    return 2;
}
```

Contoh Function 3: Function dengan parameter

```
void main(){
    print(kalikanDua(6));
}

kalikanDua(angka){
    return angka * 2;
}
```

Contoh Function 4: Pengiriman parameter lebih dari satu

```
void main(){
    print(kalikan(5,6));
}

kalikan(x, y){
    return x * y;
}
```

Contoh Function 5: Inisialisasi parameter dengan nilai default

```
void main(){
    tampilkanangka(5);
}

tampilkanangka(n1,{s1:45}){
    print(n1); //hasil akan 5 karena initalisasi 5 didalam value tampilkan
    print(s1); //hasil adalah 45 karena dari parameter diisi 45
}
```

Waspadai pengiriman parameter yang UNDEFINED!

Kita juga dapat menampung function sebagai variable dengan sebuah bentuk function yang dinamakan Anonymous Function.

```
void main(){
    print(functionPerkalian(5,6));
}
```

```
}
```

```
functionPerkalian(angka1, angka2){  
    return angka1 * angka2;  
}
```

g. Hasil dan Pembahasan:

1. Dokumentasi dalam bentuk laporan hasil implementasi function pada materi pemrograman dart.

h. Rubrik Penilaian :

No	Indikator	Skor*			
1	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan tugas	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan tugas	1	2	3	4
Jumlah skor					

Acara 10 (Pemrograman Dart II (OOP))

Materi Pembelajaran : Pemrograman Dart II
Acara Praktikum/Praktik : Pemrograman Dart II (OOP)
Tempat : *Laboratorium Multimedia Cerdas*
Alokasi Waktu : *170 Menit*

a. Capaian Pembelajaran Mata Kuliah (CPMK) :

- Mahasiswa mampu memahami pemrograman dart.
- Mahasiswa mampu mengenal dan menerapkan konsep OOP dalam pemrograman dart.

b. Penilaian Bertahap BNSP (*Skill Passport*) :

Nama Skema Sertifikasi :

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
	J.612000.021	Mengembangkan Mobile Sensor pada Mobile Computing Environment	Menjelaskan pemrograman sensor Mobile

c. Indikator Penilaian :

- Keberhasilan mahasiswa dalam memahami pemrograman dart..
- Keberhasilan mahasiswa dalam mengenal dan menerapkan konsep OOP dalam pemrograman dart.

d. Dasar Teori :

Secara singkat class dapat diartikan blueprint dari suatu objek, maksudnya adalah kita mendesain suatu objek berdasarkan class yang kita buat, sebagai contoh dalam dunia nyata, jika kita ingin membuat sebuah objek meja maka kita perlu menggambar rancangan meja tersebut, dengan demikian kita akan mulai membuat meja berdasarkan rancangan tersebut, jika rancangannya baik maka hasilnya baik, begitu juga sebaliknya, jadi class dapat kita analogikan sebagai rancangannya dan hasil dari rancangan tersebut adalah objeknya.

1. Mendefenisikan Class

Untuk membuat kelas kita gunakan keyword class yang telah disediakan oleh bahasa Dart. Perhatikan contoh berikut:

```
class Dadu {  
  
}
```

Pertama kita mulai dengan keyword class, diikuti dengan nama kelasnya yaitu Dadu, kemudian tubuh kelas diawali dengan buka kurung kurawal "{" dan diakhiri dengan tutup kurung kurawal "}" Jika sebuah kelas telah dibuat, maka kita langsung dapat menciptakan objek dari kelas tersebut, tentu jika kelas kita seperti contoh kelas Dadu, maka objek tersebut tidak dapat melakukan apapun, karena kita belum menambahkan atribut ataupun metode kedalamnya. Untuk menciptakan objek pada pemrograman Dart kita gunakan keyword new. Perhatikan contoh program berikut ini:

```
Dadu dd = new Dadu();
```

Diawali dengan nama kelas yaitu Dadu kemudian nama objeknya yaitu dd (kita bisa menentukan nama objek yang lain tidak harus dd) kemudian new diakhiri dengan nama kelas.

```
nama_kelas nama_objek = new nama_kelas();
```

2. Enkapsulasi (Pembungkusan)

Enkapsulasi (encapsulation) adalah sebuah metode untuk mengatur struktur class dengan cara menyembunyikan alur kerja dari class tersebut.

Struktur class yang dimaksud adalah property dan method. Dengan enkapsulasi, kita bisa membuat pembatasan akses kepada property dan method, sehingga hanya property dan method tertentu saja yang bisa diakses dari luar class. Enkapsulasi juga dikenal dengan istilah 'information hiding'. Dengan enkapsulasi, kita bisa memilih property dan method apa saja yang boleh diakses, dan mana yang tidak boleh diakses. Dengan menghalangi kode program lain untuk mengubah property tertentu, class menjadi lebih terintegrasi, dan menghindari kesalahan ketika seseorang 'mencoba' mengubahnya. Programmer yang merancang class bisa menyediakan property dan method khusus yang memang ditujukan untuk diakses dari luar. Pada pemrograman Dart tidak ada keyword untuk membuat atribut atau metode menjadi private ataupun public, untuk membuat atribut ataupun fungsi menjadi private, cukup tambahkan underscore "_" sebelum nama atribut ataupun metode.

3. Inheritance

Inheritance adalah konsep OOP dimana sebuah class dapat menurunkan property dan method yang dimilikinya kepada class lain. Konsep inheritance dipakai untuk memanfaatkan fitur code reuse, yakni menghindari terjadinya duplikasi kode program.

Dalam bahasa Indonesia, inheritance ini disebut juga sebagai pewarisan atau penurunan.

Konsep inheritance membuat sebuah struktur atau hierarchy class dalam kode program. Class yang akan diturunkan bisa disebut sebagai class induk (parent class), super class, atau base class.

Sedangkan class yang menerima penurunan bisa disebut sebagai class anak (child class), sub class, derived class atau heir class.

Tidak semua property dan method class induk akan diturunkan. Property dan method dengan hak akses private tidak akan diturunkan kepada class anak. Hanya property dan method dengan hak akses protected dan public saja yang bisa diakses dari class anak.

Jadi yang dimaksud dengan inheritance dari class ultramen dan godzilla adalah extends, dimana class ultramen dan godzilla mewarisi sifat dari character yang dimana character memiliki object berupa level point sehingga saat dikumpulkan di main.dart secara default godzilla dan ultramen dari mengisi dari levelPoint yang ada pada parentnya yaitu character.

4. Polymorphism

Polymorphism adalah konsep di mana suatu objek yang berbeda-beda dapat diakses melalui interface yang sama. Sebuah objek yang polymorphic dapat beradaptasi dengan metode apapun yang diimplementasikan pada objek tersebut, dan setiap class memiliki interpretasinya tersendiri terhadap interfacenya.

(SubBab) Tambahan Constructor

Constructor adalah method khusus yang akan dijalankan secara otomatis pada saat sebuah objek dibuat (instansiasi), yakni ketika perintah "new" dijalankan.

Constructor biasa digunakan untuk membuat proses awal dalam mempersiapkan objek, seperti memberi nilai awal kepada property, memanggil method internal dan beberapa proses lain yang digunakan untuk mempersiapkan objek.

e. Alat dan Bahan :

Adapun alat dan bahan meliputi:

- Kertas HVS
- Spidol Besar
- Bolpoin

f. Prosedur Kerja ;

Mengubah Prosedural ke Class

buatlah sebuah folder baru dengan nama soal1 lalu di dalamnya buat file baru yaitu soal1.dart, lalu dari sistem prosedural ini ubahlah menjadi class dan object sehingga dapat di panggil luasnya berupa class dan object

```
void main(List<String> args) {  
  double setengah, alas, tinggi;  
  setengah = 0.5;  
  alas = 20.0;  
  tinggi = 30.0;  
  
  var luasSegitiga = setengah * alas * tinggi ;  
  
  print(luasSegitiga);  
}
```

Enkapsulasi(Pembungkusan)

Buatlah folder baru di dalam Tugas-7-oop dengan nama Tugas-Enkapsulasi, Dengan menggunakan teknik enkapsulasi buatlah sebuah class yang menerima parameter untuk menghitung luas lingkaran, rule nya gunakan validasi apabila input nilai -(min) maka harus di di kalikan -1 agar nilai tetap positif, selain itu boleh menggunakan method getter dan setter atau hanya menggunakan property getter dan setter, jadi boleh dipilih salah satu.

main.dart

```
import 'lingkaran.dart';  
void main(List<String> args) {  
  //tuliskan coding disini  
}
```

lingkaran.dart

```
class Lingkaran{  
  //tuliskan coding disini  
}  
  
double get luas => // isi coding disini ;  
}
```

Inheritance

Buatlah folder baru di dalam Tugas-7-oop dengan nama Tugas-Inheritance lalu di dalam folder tersebut beri lagi file pada soal ini buatlah 5 class dan 1 main, isinya adalah

1. armor_titan.dart
2. attack_titan.dart
3. beast_titan.dart
4. human.dart
5. main.dart

6. titan.dart

pada

class 1 armor_titan memiliki object string yaitu terjang()=>>"dush.. dush..",

class 2 attack_titan memiliki object string yaitu punch()=>>"blam.. blam.."

class 3 beast_titan memiliki object string yaitu lempar()=>>"wush wush..",

class 4 human memiliki object string yaitu killAlltitan()=>>" Sasageyo ... Shinzo Sasageyo..."

selanjutnya di file titan.dart

buatlah class titan yang memiliki _powerPoint; yang akan men-get dan men-set nilai dari powerPoint masing masing turunannya

pada kelas 1 - 4 akan inheritance dari class titan, jadi otomatis akan memiliki power point setiap class nya

pada main.dart isilah value powerPoint yang ada pada 5 class dan juga tampilkan masing masing powerpoint nya jika powerPoint di bawah 5 maka akan di cetak 5 dan juga cetak masing masing object yang ada pada class masing-masing yang terdapat pada child class 1-4.

Polymorism

Buatlah folder baru di dalam Tugas-7-oop dengan nama Tugas-Polymorism lalu di dalam folder tersebut beri lagi file class dengan nama bangun_datar, segitiga, lingkaran, persegi, dan juga file main.dart untuk menjalankan classnya

pada bangun_datar akan mereturn luas dan keliling,

pada class Lingkaran akan mengextend dari bangun_datar

setelah itu akan dimasukan object nya yaitu rumus keliling dan luas ,

class persegi juga sama, adanya object luas sisi * sisi dan keliling = 4 * sisi ,

class segitiga ada luas = 0.5 * alas * tinggi, keliling = a +b + t, segitiga yang digunakan yaitu siku-siku

class lingkaran ada luas = 3.14 * jari-jari * jari-jari, keliling = 2 * pi * jari-jari

dan main.dart akan memanggil object bangun_datar.dart, selain itu juga akan memanggil class segitiga, persegi dan lingkaran dan akan mencetak luas dan keliling;

Constructor

Buatlah folder baru di dalam Tugas-7-oop dengan nama Tugas-constructor, di folder ini berisi main.dart dan juga employee.dart di dalam class employee buatlah object yang berisi id, name , departement , lalu panggil ke main.dart sebagai object yang akan bisa diisi value nya isi value bebas boleh nama pribadi teman-teman.

Jika sudah selesai dikerjakan, silahkan push hasil pekerjaan Anda ke repository Gitlab masing-masing (gunakan repository project yang sama bootcampFlutter yang digunakan folder Tugas-6-oop).

g. Hasil dan Pembahasan:

- Dokumentasi dalam bentuk laporan hasil implementasi OOP pada materi pemrograman dart.

h. Rubrik Penilaian :

No	Indikator	Skor*			
1	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan tugas	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan tugas	1	2	3	4
Jumlah skor					

Acara 11(Pemrograman Dart II (Async & Sync))

Materi Pembelajaran	: Pemrograman Dart II
Acara Praktikum/Praktik	: Pemrograman Dart II (Async & Sync)
Tempat	: <i>Laboratorium Multimedia Cerdas</i>
Alokasi Waktu	: <i>170 Menit</i>

a. Capaian Pembelajaran Mata Kuliah (CPMK) :

- Mahasiswa mampu memahami pemrograman dart.
- Mahasiswa mampu mengenal dan menerapkan konsep Asynchronous dan Synchronous dalam pemrograman dart.

b. Penilaian Bertahap BNSP (*Skill Passport*) :

Nama Skema Sertifikasi :

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
	J.612000. 021	Mengembangkan Mobile Sensor pada Mobile Computing Environment	Menjelaskan pemrograman sensor Mobile

c. Indikator Penilaian :

- Keberhasilan mahasiswa memahami pemrograman dart.
- Keberhasilan mahasiswa mengenal dan menerapkan konsep Asynchronous dan Synchronous dalam pemrograman dart.

d. Dasar Teori :

Di dalam dunia pemrograman dart terdapat dua cara dalam menjalankan program: Synchronous dan Asynchronous. Synchronous artinya program berjalan secara berurutan sedangkan Asynchronous artinya program berjalan bersama-sama.

Terkadang di dalam program yang kita buat terdapat suatu sintaks yang mengharuskan code pada baris tersebut untuk dijalankan terlebih dahulu sebelum menjalankan sintaks pada baris selanjutnya. Hal ini dikenal dengan istilah blocking. Sebaliknya non-blocking artinya program berjalan dengan mengeksekusi sintaks dari baris ke baris secara paralel (bersama-sama) .

Perhatikan contoh program di bawah ini:

```
import 'dart:async';  
void main () {  
  print("saya dijalankan pertama");  
  var timer = Timer(Duration(seconds: 3), ()=>print('saya dijalankan terakhir'));  
  print("saya dijalankan kedua");  
}
```

Jika kita jalankan program di atas, maka yang akan tampil terlebih dahulu di print adalah "saya dijalankan pertama" dan yang kedua adalah coding terakhir dan yang dicetak terakhir adalah var timer, walaupun ditulis kedua karena proses async yang terdapat pada dart.

e. Alat dan Bahan :

Adapun alat dan bahan meliputi:

- Kertas HVS
- Spidol Besar
- Bolpoin

f. Prosedur Kerja ;

Synchronous to Asynchronous

Buatlah folder baru yaitu Tugas-9-AsyncAwaitFuture, setelah itu buat file baru untuk mengerjakan soal 1 yaitu soal1.dart kemudian ketikkan coding di bawah ini, selanjutnya ubahlah coding synchronous berikut kedalam coding asynchronous dengan menggunakan async await dan future, untuk output yang diharapkan yaitu saat di compile pertama munculnya yaitu luffy, zoro, killer dan nama character one piece selanjutnya 3 detik kemudian muncul get data [done] , name 3: hilmy

Nb: coding boleh di ubah- ubah sesuai kebutuhan dan logic dari teman-teman

```
void main(List<String> args) {  
  var h = Human();  
  
  print("Luffy");  
  print("zoro");  
  print("killer");  
  h.getData();  
  print(h.name);  
}  
  
class Human {  
  String name = "nama character one piece";  
  
  void getData(){  
    name = "hilmy";  
    print("get data [done]");  
  }  
}
```

g. Hasil dan Pembahasan:

Dokumentasi dalam bentuk laporan hasil implementasi mengenal dan menerapkan konsep Asynchronous dan Synchronous pada materi pemrograman dart.

h. Rubrik Penilaian :

No	Indikator	Skor*			
1	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan tugas	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan tugas	1	2	3	4
Jumlah skor					

Acara 12 (Pemrograman Dart II (Async, Future, Await))

Materi Pembelajaran : Pemrograman Dart II
Acara Praktikum/Praktik : Pemrograman Dart II (Async, Future, Await)
Tempat : *Laboratorium Multimedia Cerdas*
Alokasi Waktu : *170 Menit*

a. Capaian Pembelajaran Mata Kuliah (CPMK) :

- Mahasiswa mampu memahami pemrograman dart.
- Mahasiswa mampu mengenal dan menerapkan konsep Async, Future dan Await dalam pemrograman dart.

b. Penilaian Bertahap BNSP (*Skill Passport*) :

Nama Skema Sertifikasi :

No	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
	J.612000.021	Mengembangkan Mobile Sensor pada Mobile Computing Environment	Menjelaskan pemrograman sensor Mobile

c. Indikator Penilaian :

- Keberhasilan mahasiswa memahami pemrograman dart.
- Keberhasilan mahasiswa mengenal dan menerapkan konsep Async, Future dan Await dalam pemrograman dart.

d. Dasar Teori :

Future merupakan salah satu keyword yang disediakan oleh dart yang seperti namanya yaitu masa depan, jadi keyword ini gunanya untuk mengembalikan masa depan atau mengembalikan nilainya untuk waktu yang akan datang
contoh penggunaan future

```
void main() {  
  fetchUserOrder();  
  print('Ambil Datanya');  
}  
  
Future<void> fetchUserOrder() {  
  return Future.delayed(Duration(seconds: 2),  
    () => print('Selamat datang Peserta bootcamp flutter'));  
}
```

Bekerja dengan async dan await

Keyword Async dan Await menyediakan cara deklaratif untuk mendefinisikan fungsi asynchronous dan menggunakan valuenya. dalam menggunakan async dan await ingat pedoman dasar ini :

1. sebelumn menggunakan asynchronous terlebih dahulu tambahkan keyword async sebelum body
2. keyword await akan berfungsi apabila dia berada dalam fungsi async
3. dan berikut ini merupakan contoh yang mengkonversi dari fungsi synchronous ke dalam asynchronous

```
void main() async { ... }
```

jika fungsi memiliki tipe pengembalian yang di deklarasikan, maka perlu di perbaharui type menjadi `Future<T>`, dimana T adalah tipe nilai yang dikembalikan oleh fungsi, jika fungsi pengembaliannya tidak secara eksplisit maka tipe data akan menjadi `Future<void>`

contoh :

```
Future<void> main() async { ... }
```

Sekarang Anda memiliki fungsi async, Anda dapat menggunakan kata kunci await untuk menunggu hingga Future selesai:

```
print(await createOrderMessage());
```

Contoh Synchronous

```
String createOrderMessage() {  
  var order = fetchUserOrder();  
  return 'Your order is: $order';  
}  
  
Future<String> fetchUserOrder() =>  
  // Imagine that this function is  
  // more complex and slow.  
  Future.delayed(  
    Duration(seconds: 2),  
    () => 'Large Latte',  
  );  
  
void main() {  
  print('Fetching user order...');  
  print(createOrderMessage());  
}
```

Contoh Asynchronous

```
Future<String> createOrderMessage() async {  
  var order = await fetchUserOrder();  
  return 'Your order is: $order';  
}  
  
Future<String> fetchUserOrder() =>  
  // Imagine that this function is  
  // more complex and slow.  
  Future.delayed(  
    Duration(seconds: 2),  
    () => 'Large Latte',  
  );  
  
Future<void> main() async {  
  print('Fetching user order...');  
  print(await createOrderMessage());  
}
```

contoh asynchronous akan berbeda dalam tiga hal:

Jenis return/kembalian untuk `createOrderMessage ()` berubah dari `String` ke `Future <String>`.

keyword `async` muncul sebelum body fungsi untuk `createOrderMessage ()` dan `main ()`.

keyword `await` muncul sebelum memanggil fungsi Asynchronous `fetchUserOrder ()` dan `createOrderMessage ()`.

Handling Error

untuk menangani handling error function `async` gunakan, `try-catch` jadi apabila sukses akan masuk ke `try` dan jika error akan di tangani oleh `catch` dan berikut ini merupakan contoh struktur coding `try catch` :

```
try {
  var order = await fetchUserOrder();
  print('Awaiting user order...');
} catch (err) {
  print('Caught error: $err');
}
```

Try-Catch contoh di dalam program

```
Future<void> printOrderMessage() async {
  try {
    var order = await fetchUserOrder();
    print('Awaiting user order...');
    print(order);
  } catch (err) {
    print('Caught error: $err');
  }
}

Future<String> fetchUserOrder() {
  // Imagine that this function is more complex.
  var str = Future.delayed(
    Duration(seconds: 4),
    () => throw 'Cannot locate user order');
  return str;
}

Future<void> main() async {
  await printOrderMessage();
}
```

Bekerja dengan synchronous dan asynchronous

dengan menggabungkan 2 blok ini kita dapat membuat ui kita menjadi lebih interaktif, sehingga lebih memudahkan kita dalam menyajikan data yang interaktif, contoh penggunaan synchronous dan asynchronous sebagai berikut :

```
void main(List<String> args) async{
  var t = Titan(); // inialisasi t = object dari class titan

  print("zeke"); // mencetak zeke
  print(t.name); // akan mencetak string name yang pertama yaitu eren yeger
  await t.getName(); // masuk kedalam async await untuk delayed object di dalamnya
  //dan akan mencetak get nem [done]
  print(t.name); //mencetak grisha yeger karena sudah di masukan kedalam async nya
  print("rener"); // akan me cetak rener
}

class Titan{
  String name = "Eren Yeger"; // mengisi value name eren yeger
  Future<void> getName()async{ // masuk kedalam Future dan object get name dan inisialisai async
    await Future.delayed(Duration(seconds: 3)); // memberi delayed pada object selama 3 detik
    name = "grisha"; //set name grisha
    print("get name [done]"); //cetak prin get name done
  }
}
```

Bekerja dengan future dan delayed tanpa async await

dengan menggunakan method future kita juga dapat membuat sebuah asynchrobnous dengan sama sama lebih mudah, berikut ini adalha contoh nya

```
Future delayedPrint(int seconds, String message){
  final duration = Duration(seconds : seconds);
  return Future.delayed(duration).then((value) => message);
}

Run | Debug
main(List<String> args){
  print("roger");
  delayedPrint(2,"pirates"). then((status){
    print(status);
  });
  print("is");
}
```

Bekerja dengan async, await, future

dengan menggabungkan ketiganya kita dapat membuat aplikasi kita jauh lebih powerfull lagi dibanding kita harus bekerja dengan beberapa method sendiri- sendiri, untuk contoh penggunaanya sebagai berikut :

```
void main(List<String> args) async {
  print("Persiapan. mulai");
  print(await line());
  print(await line2());
}

Future<String> line() async {
  String greeting = "pagiku cerah matahari bersinar...";
  return await Future.delayed(Duration(seconds: 5), () => (greeting));
}

Future<String> line2() async {
  String greeting = "kugendong tas merahaku, di pundak .....";
  return await Future.delayed(Duration(seconds: 3), () => (greeting));
}
```

e. Alat dan Bahan :

Adapun alat dan bahan meliputi:

- Kertas HVS
- Spidol Besar
- Bolpoin

f. Prosedur Kerja ;

Future delayed

buatlah file baru dengan nama soal2.dart kemudian dengan menggunakan function future delayed buatlah output dari coding ini menjadi life is never flat, tidak perlu menggunakan async await

```
main(List<String> args){  
  print("Life");  
  print("never flat");  
  print("is");  
}
```

Async, Await and future

buatlah file baru dengan nama soal3.dart kemudian dengan menggunakan async await dan future ubahlah string berikut menjadi sebuah subtitle lagu, jadi awal masuk akan mencetak "Ready. sing" setelah itu ada jeda 5 detik kemudian akan muncul line () yang akan mencetak pernahkah kau merasa, setelah mencetak line() kemudian ada jeda waktu 3 detik untuk mencetak line2() yang berisi pernahkah kau merasa setelah mencetak line2() kemudian ada jeda waktu 2 detik untuk mencetak line3() yang berisi pernahkah kau merasa, dan terakhir yaitu memunculkan line4() dengan jeda 1 detik dari line 3 ke line 4 yang berisi, hatimu hampa pernahkah kau merasa hati mu kosong

```
void main(List<String> args) {
    print("Ready. Sing");
    line();
    line2();
    line3();
    line4();
}

line(){
    print("perhatikan kau merasa");
}
line2(){
    print("perhatikan kau merasa....");
}

line3(){
    print("perhatikan kau merasa");
}
line4(){
    print("Hatimu hampa, perhatikan kau merasa hati mu kosong....");
}
```

g. Hasil dan Pembahasan:

Dokumentasi dalam bentuk laporan hasil implementasi mengenal dan menerapkan konsep Async, Future dan Await pada materi pemrograman dart.

h. Rubrik Penilaian :

No	Indikator	Skor*			
1	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan tugas	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan tugas	1	2	3	4
Jumlah skor					